CS 555: Homework 1

Sagnik Saha*, Lei An†

February 19, 2026

Problem 1

Part (a)

n_{steps}	L^2 error	Successive error ratio
25	7.686488e-02	-
50	3.869765e-02	1.986293e+00
100	1.941534e-02	1.993148e+00
200	9.724347e-03	1.996570e+00
400	4.866376e-03	1.998273e+00
800	2.434271e-03	1.999111e+00
1600	1.217437e-03	1.999504e+00

Table 1: Euler Backward (EB) with $N = 1500$ and $u_0(x) = \sin(\pi x)$.

Based on the tables above, it is clear that EB is $\mathcal{O}(\Delta t)$ accurate globally since the error ratio remains at $\approx 2^1 = 2$, while CN is $\mathcal{O}(\Delta t^2)$ accurate globally, since the error ratio remains at $\approx 2^2 = 4$, as the step size is halved.

Part (b)

Now we switch to the constant initial condition $u_0(x) = \pi/4$. The tables are shown below:

From these tables, we note that the successive error ratio of EB remains the same, at ≈ 2, indicating that the GTE is still $\mathcal{O}(\Delta t)$. On the other hand, for CN the successive error ratio drops sharply from ≈ 4 earlier to

*sagniks2@illinois.edu
†leian2@illinois.edu

n_{steps}	L^2 error	Successive error ratio
25	1.025831e-03	-
50	2.563136e-04	4.002251e+00
100	6.401055e-05	4.004239e+00
200	1.594024e-05	4.015658e+00
400	3.925755e-06	4.060427e+00
800	9.342016e-07	4.202257e+00
1600	2.422498e-07	3.856356e+00

Table 2: Crank-Nicholson (CN) with $N = 1500$ and $u_0(x) = \sin(\pi x)$.

n_{steps}	L^2 error	Successive error ratio
25	7.686525e-02	-
50	3.869766e-02	1.986302e+00
100	1.941530e-02	1.993153e+00
200	9.724300e-03	1.996576e+00
400	4.866325e-03	1.998284e+00
800	2.434218e-03	1.999133e+00
1600	1.217385e-03	1.999548e+00

Table 3: EB with $N = 1500$ and $u_0(x) = \pi/4$.

≈ 1.1, indicating that now the (L^2) error depends *very weakly* on Δt. We can understand this better by plotting $u(x, t_0)$ for various values of t_0 as n_{steps} is successively doubled. The results are shown in Figs. 1 and 2 for EB and CN, respectively. From these figures, we see that CN overshoots the boundaries significantly, while EB does not. See also the tables 3 and 4.

The reason for this difference between EB and CN (in the case of the square wave) is that the former is L-stable while the latter is not. This is because, as we will see in Problem 2, the growth factor G_{CN} for Crank-Nicholson has the value -1 as $\lambda\Delta t \to -\infty$, and hence, has trouble dealing with stiff systems.

Part (c)

Next, we use BDF2. The general expression that we need to implement is,

$$(3I + 2\nu A\Delta t)u^k = I(4u^{k-1} - u^{k-2}), \quad k \in [2, n_{steps}]. \tag{1}$$

We bootstrap the procedure using Euler Backward as,

$$(I + A\nu\Delta t)u^k = u^{k-1}, \quad k = 1. \tag{2}$$

n_{steps}	L^2 error	Successive error ratio
25	2.490516e+00	-
50	2.089526e+00	1.191904e+00
100	1.751750e+00	1.192822e+00
200	1.466818e+00	1.194252e+00
400	1.226047e+00	1.196380e+00
800	1.022115e+00	1.199520e+00
1600	8.487955e-01	1.204194e+00

Table 4: CN with $N = 1500$ and $u_0(x) = \pi/4$.

From Eq. (1), we see that we must keep track of *two* previous iterations of u; this is the source of the main modification to the provided code. In addition, the EB bootstrapping requires an *if* statement for $k = 1$.

Tables 5 and 6 show the L^2 errors and successive error ratios for $u_0(x) = \sin(\pi x)$ and $u_0(x) = \pi/4$, respectively. For both initial conditions, we see that the successive error ratio remains very close to 4, as expected from a scheme with $\mathcal{O}(\Delta t^2)$ GTE.

Figs. 3 and 4 show the plots for $u(x, t_0)$ for various values of t_0 as n_{steps} is successively doubled, for $u_0(x) = \sin(\pi x)$ and $u_0(x) = \pi/4$, respectively. Unlike when using CN, the initial condition $u_0(x) = \pi/4$ does not produce any ringing near the boundaries, which means that BDF2 is L-stable.

n_{steps}	L^2 error	Successive error ratio
25	5.997632e-04	-
50	1.452498e-04	4.129184e+00
100	3.607271e-05	4.026584e+00
200	9.061195e-06	3.981010e+00
400	2.332794e-06	3.884267e+00
800	6.679974e-07	3.492220e+00

Table 5: BDF2 (bootstrapped with EB) with $N = 1500$ and $u_0(x) = \sin(\pi x)$.

Extra Credit

The general solution to the heat equation in the domain $x \in [0, 1]$,

$$u_t = \nu u_{xx}, \quad u(0, t) = u(1, t) = 0, \quad u(x, 0) = u^0(x), \tag{3}$$

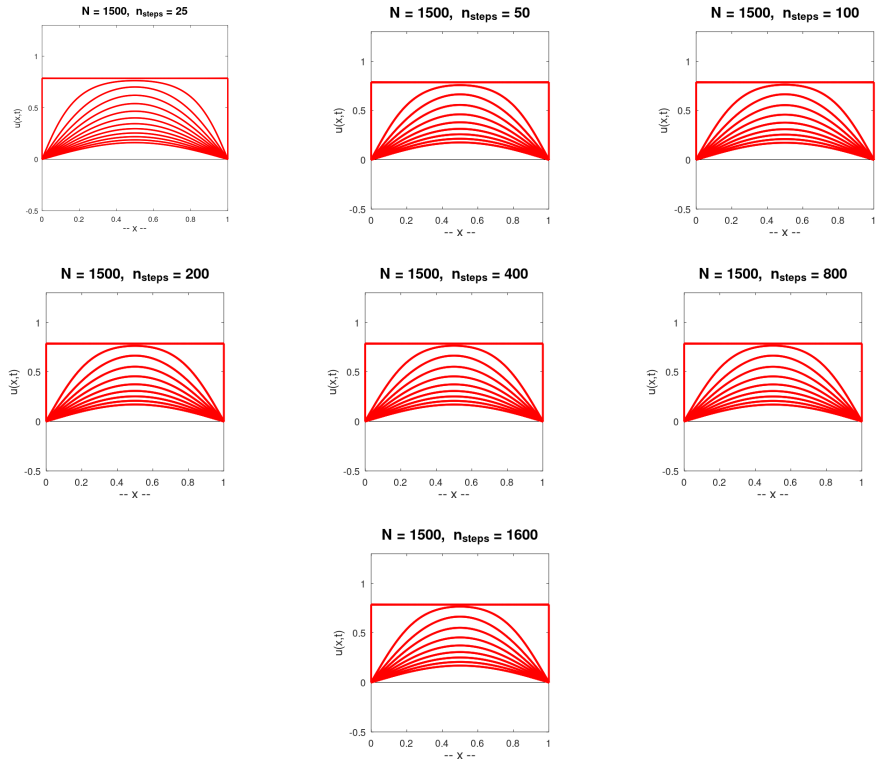

Figure 1: $u(x, t_0)$ for various t_0 values as the number of time steps, n_{steps}, is successively doubled from 25 to 1600, using EB. Here $u_0(x) = \pi/4$.

is given as

$$u(x,t) = \sum_{k=1}^{\infty} c_k e^{-k^2\pi^2\nu t} \sin(k\pi x), \quad c_k = 2\int_0^1 dx u^0(x) \sin(k\pi x). \tag{4}$$

If $u^0(x) = \sin(\pi x)$, then we have $c_k = \delta_{1k}$. For $u^0(x) = \pi/4$, we have,

$$c_k = \frac{1}{2k}[1 - \cos(k\pi)] \implies c_k = \begin{cases} 1/k & k \text{ odd}, \\ 0 & k \text{ even}. \end{cases} \tag{5}$$

As such, we can write down the full solution to the square wave as,

$$\boxed{u_{sw}(x,t) = \sum_{k=1,3,5,\ldots}^{\infty} \frac{1}{k} e^{-k^2\pi^2\nu t} \sin(k\pi x).} \tag{6}$$

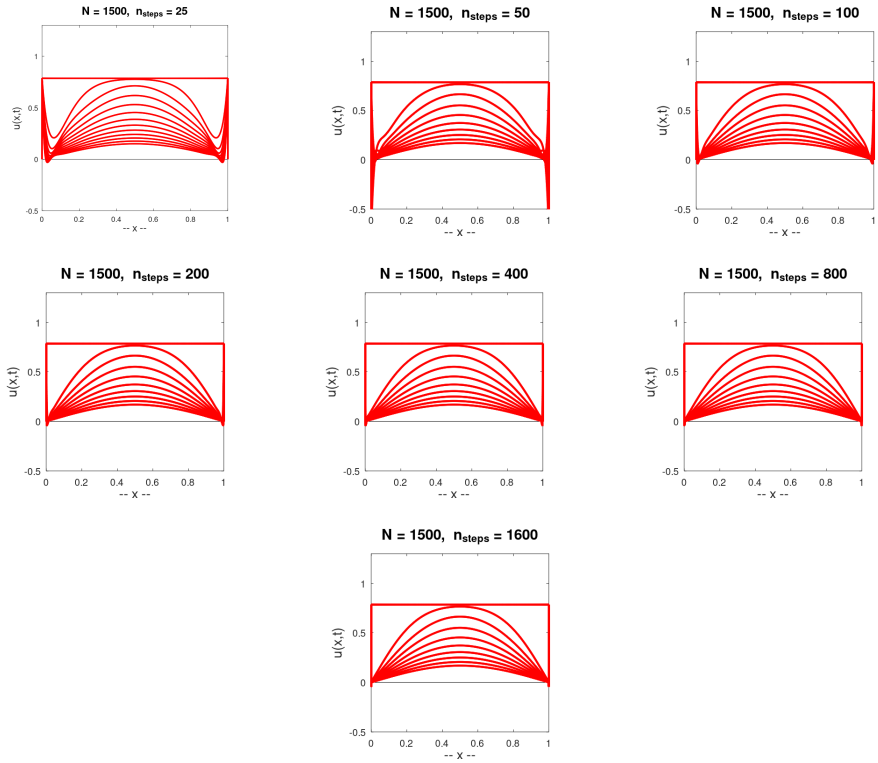

Figure 2: $u(x, t_0)$ for various t_0 values as the number of time steps, n_{steps}, is successively doubled from 25 to 1600, using CN. Here $u_0(x) = \pi/4$. Note the overshoot at the boundaries.

Note that the $k = 1$ term corresponds to the solution where $u^0(x)$ is the $k = 1$ eigenmode. As such, the next leading order term (at $t = 1$) occurs for $k = 3$:

$$= \frac{1}{3} e^{-(3\pi)^2(0.2)(1)} \underbrace{\sin(3\pi x)}_{\sim 1} = 6.42 \cdot 10^{-9}. \tag{7}$$

We compare it to the $k = 1$ term (at $t = 1$):

$$\frac{0.1389}{6.42 \cdot 10^{-9}} = 7.2 \cdot 10^6. \tag{8}$$

Thus, the sub-leading term is smaller by a factor of (more than) a million (10^6) at $t = 1$. This error is (much) **smaller** than the finite difference results of the previous section (even when we exclude CN, which was terrible for $u^0(x) = \pi/4$ due to reasons of L-stability).

n_{steps}	L^2 error	Successive error ratio
25	5.997064e-04	-
50	1.451928e-04	4.130415e+00
100	3.601819e-05	4.031095e+00
200	9.006817e-06	3.998993e+00
400	2.276327e-06	3.956732e+00
800	6.046393e-07	3.764769e+00

Table 6: BDF2 (bootstrapped with EB) with $N = 1500$ and $u_0(x) = \pi/4$.

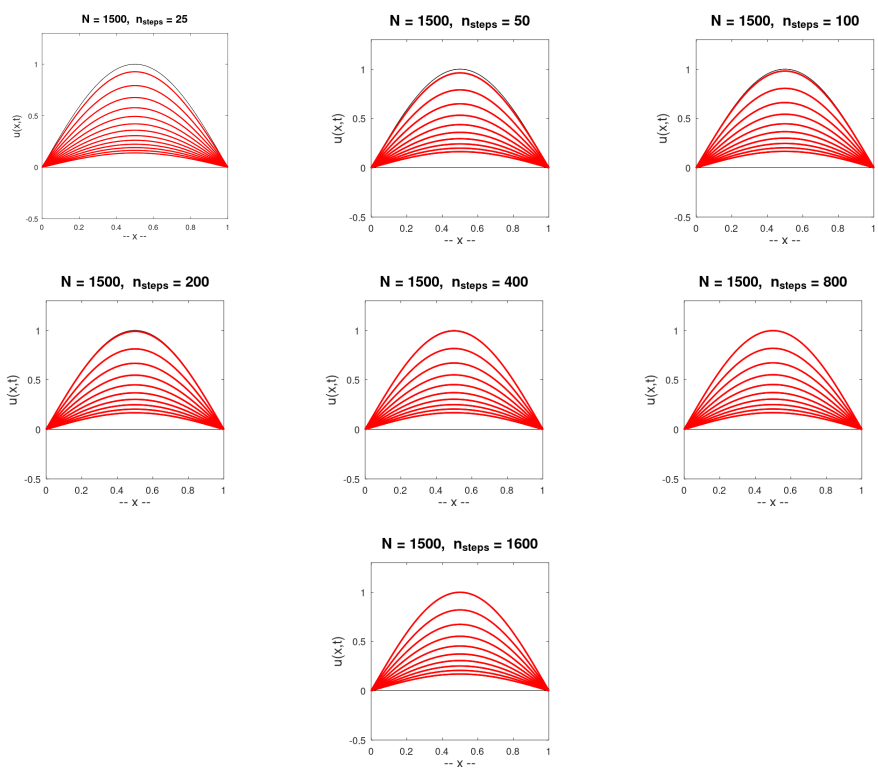

Figure 3: $u(x, t_0)$ for various t_0 values as the number of time steps, n_{steps}, is successively doubled from 25 to 1600, using BDF2 (bootstrapped with EB). Here we use $u_0(x) = \sin(\pi x)$.

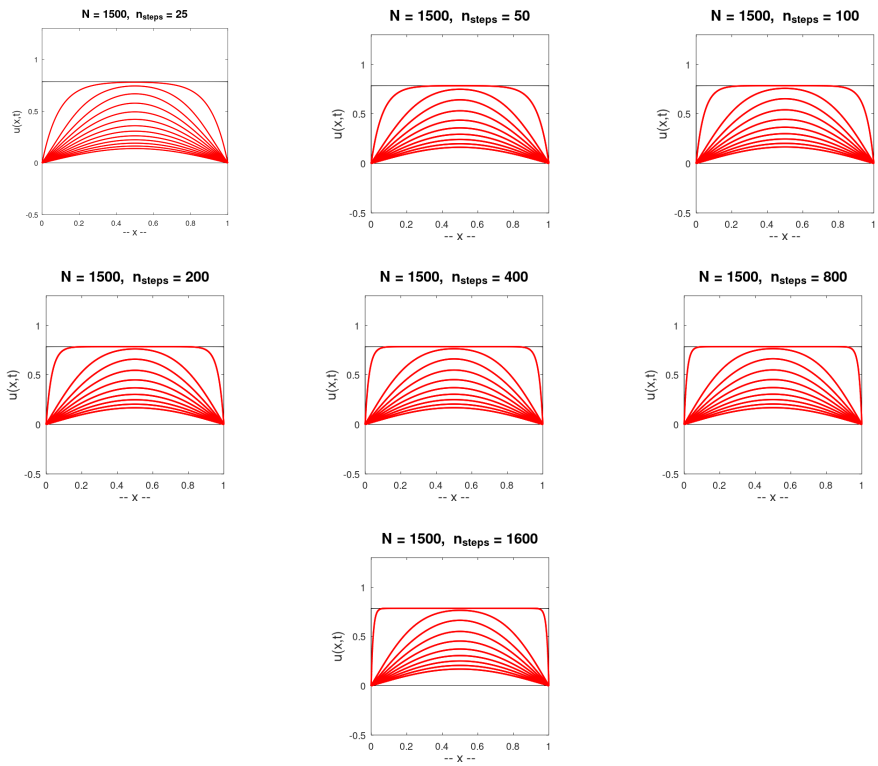

Figure 4: $u(x, t_0)$ for various t_0 values as the number of time steps, n_{steps}, is successively doubled from 25 to 1600, using BDF2 (bootstrapped with EB). Here we use $u_0(x) = \pi/4$.

Problem 2

Part (i)

The growth factor for Crank-Nicholson is given as,

$$G_{CN} = \frac{1 + \lambda\Delta t/2}{1 - \lambda\Delta t/2} \tag{9}$$

If $\lambda\Delta t/2 \to -\infty$, then Eq. (9) can be expressed as,

$$\sim \lim_{x\to\infty} \frac{1-x}{1+x} \implies \lim_{x\to\infty} \frac{0-1}{0+1} = \boxed{-1} \quad \text{(using L'Hopital's Rule).} \tag{10}$$

Thus, in the limit $\lambda\Delta t/2 \to -\infty$, $\boxed{G_{CN} \to -1.}$

Part (ii)

The growth factors for the various time steppers are:

$$\begin{aligned} G_{exact} &= e^{\lambda\Delta t} \\ G_{EF} &= 1 + \lambda\Delta t \\ G_{EB} &= (1 - \lambda\Delta t)^{-1} \\ G_{CN} &= \frac{1 + \lambda\Delta t/2}{1 - \lambda\Delta t/2} \end{aligned} \tag{11}$$

The plot (for $-10 \leq \lambda\Delta t \leq 0$) comparing the growth factors for the various time steppers is shown in Fig. 5.

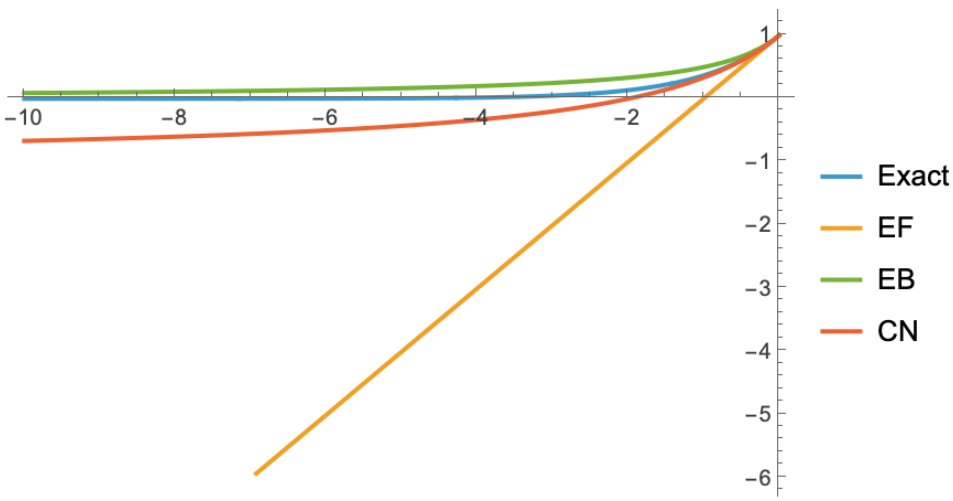

Figure 5: Growth factors for the following time steppers: analytical (exact), Euler Forward (EF), Euler Backward (EB), and Crank-Nicholson (CN). The horizontal axis represents $\lambda \Delta t$.

Problem 3

Part (i)

Implementation

Compared to the original *heat2d.m*, we make the following update:

```
p=symamd(HL);
L = chol(HL(p,p),'lower');
for istep=1:nstep; time=istep*dt;
  rhs = HR*u; %Compute RHS as usual.
  rhs=rhs(p); %Once computed, permute the RHS.
  u = L'\( L \rhs);
  u(p) = u;
end;
```

Note that we compute the permutation vector p w.r.t HL. Also, we must be careful about how to compute the right hand side in each step - it is easiest to compute it *without* any permutation first, and then permute it in the following step.

Timing Comparison

We now compare the timings for the original algorithm, versus the permuted algorithm. We (arbitrarily) choose $\Delta t = 0.04s$, and vary the number of spatial points N to compare the two algorithms. The results are shown in Tables 7 and 8, for the cases without and with the permutation, respectively.

From these tables, we see that without the permutation, the time taken for the script to run blows up very quickly with N. In fact, the $N = 640$ case did not even converge in a reasonable amount of time, as seen in Table 7.

Part (ii)

Implementation

The problem text tells us that we must solve a system of the following form,

$$H_{ly}H_{lx}u^l = H_{ry}H_{rx}u^{l-1}. \tag{12}$$

N	Runtime (s)	Successive Ratio
40	2.3018e-02	-
80	1.4951e-01	6.4953
160	1.4574e+00	9.7478
320	2.0624e+01	13.7659
640	?	?

Table 7: CN algorithm (without permutation) runtimes. We choose $\Delta t = 0.04s$ for all the runs, and vary the number of spatial points N. The question marks denote that for $N = 640$, the script did not converge within a reasonable amount of time.

N	Runtime (s)	Successive Ratio
40	1.6752e-02	-
80	6.0615e-02	3.6184
160	2.7623e-01	4.5571
320	1.3622e+00	4.9314
640	7.5804e+00	5.5648

Table 8: CN algorithm (with permutation) runtimes. We choose $\Delta t = 0.04s$ for all the runs, and vary the number of spatial points N. With the permutation, the $N = 640$ case converges fairly quickly, unlike the unpermuted case, as seen in Table 7.

As in lecture, for convenience, we neglect the q (source) terms. Using the fact that,

$$\begin{aligned} H_{ry} &= T_{ry} \otimes I_x \\ H_{rx} &= I_y \otimes T_{rx}, \end{aligned} \tag{13}$$

where we define,

$$\begin{aligned} T_{ry} &= I_y - \frac{\nu \Delta t}{2} A_y \\ T_{rx} &= I_x - \frac{\nu \Delta t}{2} A_x, \end{aligned} \tag{14}$$

we can re-arrange the right side of Eq. (12) as follows:

$$\begin{aligned} &= (T_{ry} \otimes I_x)(I_y \otimes T_{rx}) u^{l-1} \\ &= (T_{ry} \otimes T_{rx}) u^{l-1} \\ &= \boxed{T_{rx} u^{l-1} T_{ry}^T.} \end{aligned} \tag{15}$$

Note that here we are assuming that u^{l-1} is a matrix, and not a lexicographically ordered array. Then, Eq. (12) takes the form

$$H_{ly}H_{lx}u^l = T_{rx}u^{l-1}T_{ry}^T \equiv R. \tag{16}$$

This can be simplified in two successive steps:

1. Using the fact that for $H_{ly}u = b \implies U = (T_y^{-1}B^T)^T$, we get:

$$H_{lx}u^l = \underbrace{[T_{ly}^{-1}R^T]}_{X}{}^T \equiv X^T, \tag{17}$$

   where X is obtained (in Matlab/Octave) as,

$$X = T_{ly}\backslash R^T. \tag{18}$$

2. Once we have X, it is straightforward to solve for u^l as,

$$u^l = T_{lx}\backslash X^T. \tag{19}$$

Results

1. **Scalability as N is changed**: As in part (i), we (arbitrarily) choose $\Delta t = 0.04s$, and vary the number of spatial points N to compare the algorithms. We compare our results to the original script, and the permuted algorithm, as shown in Tables 7 and 8.

N	Runtime (s)	Successive Ratio
40	1.7120e-02	-
80	1.1636e-02	0.6797
160	4.1637e-02	3.5783
320	1.4335e-01	3.4428
640	6.3960e-01	4.4618
1280	2.5889e+00	4.0477
2560	1.1238e+01	4.3408

Table 9: CN algorithm (using the ADI modification) runtimes. We choose $\Delta t = 0.04s$ for all the runs, and vary the number of spatial points N. Compare this to Tables 7 and 8.

Compared to the permuted algorithm (Table 8), the ADI method is almost an *order of magnitude* faster (approximately). In fact, the *memory ran out*

while trying to compute the $N = 1280, 2560$ cases using the permuted algorithm, which is not the case for the ADI implementation.

2. **Accuracy as Δt is changed for fixed N**: Next we we plot the error for the three algorithms as Δt is varied for fixed N. Here, the error is defined with respect to the *exact solution*, as defined in the starter code given to us. We set $N = 80$ to ensure that we get results in a reasonable amount of time. The results are shown in Tables 10, 11, and 12 for the original, permuted, and ADI cases, respectively. We see that all three methods have the same GTE of $\mathcal{O}(\Delta t^2)$. Thus, the ADI implementation has the advantage of being much faster and less memory intensive than the other two methods.

Δt	Error	Successive ratio of errors
0.08	7.5506e-02	-
0.04	1.9095e-02	3.9542e+00
0.02	4.7873e-03	3.9887e+00
0.01	1.1977e-03	3.9972e+00
0.005	2.9947e-04	3.9993e+00
0.0025	7.4870e-05	3.9998e+00
0.00125	1.8718e-05	4.0000e+00

Table 10: Errors with respect to the exact solution for $N = 80$ spatial steps, as the time step Δt is successively halved, for the **original CN algorithm**. We see that the successive ratio, as Δt is halved, is ≈ 4, which is consistent with CN being $\mathcal{O}(\Delta t^2)$ accurate globally.

Δt	Error	Successive ratio of errors
0.08	7.5506e-02	-
0.04	1.9095e-02	3.9542e+00
0.02	4.7873e-03	3.9887e+00
0.01	1.1977e-03	3.9972e+00
0.005	2.9947e-04	3.9993e+00
0.0025	7.4870e-05	3.9998e+00
0.00125	1.8718e-05	4.0000e+00

Table 11: Errors with respect to the exact solution for $N = 80$ spatial steps, as the time step Δt is successively halved, for the **permuted CN algorithm**. We see that the successive ratio, as Δt is halved, is ≈ 4, which is consistent with CN being $\mathcal{O}(\Delta t^2)$ accurate globally.

Δt	Error	Successive ratio of errors
0.08	5.5435e-02	-
0.04	1.3956e-02	3.9722e+00
0.02	3.4949e-03	3.9931e+00
0.01	8.7411e-04	3.9983e+00
0.005	2.1855e-04	3.9996e+00
0.0025	5.4639e-05	3.9999e+00
0.00125	1.3660e-05	4.0000e+00

Table 12: Errors with respect to the exact solution for $N = 80$ spatial steps, as the time step Δt is successively halved, for the **ADI CN algorithm**. We see that the successive ratio, as Δt is halved, is ≈ 4, which is consistent with CN being $\mathcal{O}(\Delta t^2)$ accurate globally.

Extra extra credit

In lecture, we showed that for BDF3,

$$\underbrace{\left[I + \frac{6}{11}\nu\Delta t A_{2D}\right]}_{H_L} u^l = \frac{1}{11}(18u^{l-1} - 9u^{l-2} + 2u^{l-3}) + \underbrace{\mathcal{O}(\Delta t^4)}_{\text{LTE}}, \tag{20}$$

where we define,

$$\begin{aligned} I &= I_y \otimes I_x \\ A_{2D} &= I_y \otimes A_x + A_y \otimes I_x. \end{aligned} \tag{21}$$

Technical Note: The code provided incorporates ν into the defintion of A_{2D}, which we do not do here.

Next, it is easy to show that,

$$H_L = H_{lx}H_{ly} - \left(\frac{6\nu\Delta t}{11}\right)^2 (A_y \otimes A_x), \tag{22}$$

where we define,

$$\begin{aligned} H_{lx} &= I + \frac{6\nu\Delta t}{11}(I_y \otimes A_x) = I_y \otimes \underbrace{\left[I_x + \frac{6\nu\Delta t}{11}A_x\right]}_{T_l x} \\ H_{ly} &= I + \frac{6\nu\Delta t}{11}(A_y \otimes I_x) = \underbrace{\left[I_y + \frac{6\nu\Delta t}{11}A_y\right]}_{T_l y} \otimes I_x. \end{aligned} \tag{23}$$

Note that we used the fact that $(I_y \otimes A_x)(A_y \otimes I_x) = (A_y \otimes A_x)$.

Finally, note that using the Taylor expansion formula,

$$f(x_0 + h) = \sum_{n=0}^{\infty} \frac{h^n}{n!} f^{(n)}(x_0), \tag{24}$$

we can show that

$$u^l = 2u^{l-1} - u^{l-2} + \mathcal{O}(\Delta t^2). \tag{25}$$

(Simply use Eq. (24) with $x_0 = t^l$, and $h = -\Delta t$ and $-2\Delta t$, respectively).

Substituting Eqs. (22) and (25) into Eq. (20), we find

$$H_{lx}H_{ly}u^l - \left(\frac{6\nu\Delta t}{11}\right)^2 (A_y \otimes A_x)(2u^{l-1} - u^{l-2}) + \underbrace{\left(\frac{6\nu\Delta t}{11}\right)^2 \mathcal{O}(\Delta t^2)}_{\mathcal{O}(\Delta t^4)}$$
$$= \frac{1}{11}(18u^{l-1} - 9u^{l-2} + 2u^{l-3}) + \mathcal{O}(\Delta t^4).$$

The $\mathcal{O}(\Delta t^4)$ term on the left side can now be absorbed into the term of the same order on the right side. Doing so and re-arranging, we obtain

$$\boxed{H_{lx}H_{ly}u^l = \left(\frac{6\nu\Delta t}{11}\right)^2 A_x(2u^{l-1} - u^{l-2})A_y^T + \frac{1}{11}(18u^{l-1} - 9u^{l-2} + 2u^{l-3}) + \mathcal{O}(\Delta t^4).} \tag{26}$$

Note that we used the identity $(A \otimes B)u = BUA^T$. Thus, Eq. (26) is the ADI implementation for BDF3. Please see the relevant Octave script for the implementation.

Finally, as a sanity check, let us examine how the errors change as Δt is halved for a fixed number of spatial points N. Since the GTE is $\mathcal{O}(\Delta t^3)$ (since the LTE is $\mathcal{O}(\Delta t^4)$), we expect the successive error ratios to be (very close to) $2^3 = 8$. This is indeed what we see in Table 13, where we present the errors and successive error ratios for $N = 80$ spatial points.

Δt	L^2 error	Successive error ratio
8.0000e-02	9.1556e-02	-
4.0000e-02	9.5769e-03	9.5601e+00
2.0000e-02	1.0905e-03	8.7818e+00
1.0000e-02	1.3008e-04	8.3838e+00
5.0000e-03	1.5883e-05	8.1895e+00
2.5000e-03	1.9624e-06	8.0941e+00
1.2500e-03	2.4386e-07	8.0469e+00

Table 13: ADI implementation of BDF3, for fixed number of spatial points $N = 80$, as the time step Δt is succesively halved. As expected, the successive error ratios are very close to $2^3 = 8$, since BDF3 has GTE of $\mathcal{O}(\Delta t^3)$.